



CPE 334 Introduction to Software Engineering in the Age of AI Agents

Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 2. TokTickIT Requester Ticketing MVP with UI Foundation.

Score: ____ / 60

Instructors:	Assoc. Prof. Suthep Madarasmi, Ph.D. (Jogie/โจ๊ก)	(suthep.mad@kmutt.ac.th)
	Aj. Piyanit Ua-areemit, Ph.D (Toey)	(piyanit.wep@kmutt.ac.th)
	Aj. Santawat Thanyadit Ph.D. (Job)	(santawat.than@kmutt.ac.th)
TAs:	Rachawipa Katippatee (Bom)	(rachawipa.kati@gmail.com)
	Kantapat Suwannahong (Bump)	(kantapat.suwan@kmutt.ac.th)
	Rattanachote Petpansri (Loogmoo)	(rattanachote.petpa@kmutt.ac.th)
	Prapatsorn Sangrod (Noon)	(prapatsorn.sangr@kmutt.ac.th)
	Supachok Deetaweesukh (Tik)	(jedsadaporn.pann@mail.kmutt.ac.th)

1. Software Product Increment in Lab 2

Lab 2 builds the Requester-facing application using a temporary Development Requester identity for “user login” to simulate multi-user ownership before Lab 3 introduces full authentication. By the end of this sprint, a Requester can:

- create an IT support ticket;
- upload permitted supporting attachments;
- receive a unique Ticket Number;
- view their own tickets in My Tickets;
- search, filter, sort, and page through their tickets;
- open a Ticket Detail screen;
- inspect ticket information and attachments;
- add a permitted attachment to an existing ticket; and
- remove one of their own permitted attachments using the required soft-removal rules.

Ticket Details

Ticket No.: TKT-2025-001234

Ticket Date: May 12, 2025 09:14 AM

Category: Hardware

Related System: Corporate Laptop

Requester: Jennifer Anderson

Requested Priority: Medium

IT Priority: Medium

Current Status: In Progress

Ticket Owner: Michael Brown (IT Support)

Summary: Laptop battery drains quickly

Description: My laptop battery is draining much faster than usual even when the system is idle. This started happening after last week's Windows update.

Resolution Summary: No resolution summary available yet.

Public Comments (3) | Attachments (2) | Service Actions (1) | Event Log (6)

Comments:

- Jennifer Anderson (Requester)** - May 13, 2025 11:45 AM: Thank you for the update. Please let me know if you need any additional information.
- Michael Brown (IT Support)** - May 13, 2025 10:30 AM: We are investigating the issue on your device. We'll update you shortly.
- Jennifer Anderson (Requester)** - May 12, 2025 09:20 AM: Just adding that this issue occurs even when I close all applications.

Figure 1. Illustrative TokTickIT Ticket Detail screen using the Zen Green template design language. Lab 2 implements the Requester-facing Create Ticket, My Tickets, Ticket Detail, and Attachment functions. Later labs will add role-specific IT Staff controls, communication, workflow, and Actions Taken.

2. Lab 2 Learning Outcomes

- translate an incomplete stakeholder request into a clear, testable sprint engineering specification;
- apply Spec-Driven Development to define scope, business rules, UI behavior, data requirements, API contracts, acceptance criteria, and Definition of Done;
- apply Test DD and TDD to plan, implement, and trace unit, API, UI, responsive, visual, and end-to-end tests;
- design and justify a coherent full-stack solution across frontend, backend, API, and database layers;
- design reusable and responsive UI components that follow a consistent visual specification;
- identify and test validation, failure, boundary, ownership, and state-transition scenarios;
- use GitHub Issues, feature branches, Pull Requests, peer review, and staged integration as part of a disciplined engineering workflow;
- use an AI coding agent as an engineering assistant while retaining responsibility for specifications, code, tests, and design decisions; and

- evaluate completion using traceable evidence rather than accepting an AI agent's claim that the work is done.

3. Lab 2 Request from Stakeholder

“The IT department is ready to receive real support requests. Create a professional and responsive Requester (end user) facing ticketing experience. A Requester must be able to describe a problem, select its category and related system, indicate the requested priority, attach supporting evidence, and submit the ticket.

After submission, the Requester must be able to locate the ticket in My Tickets, search and filter their own tickets, open the Ticket Detail screen, and inspect or manage permitted attachments. The system must generate the official Ticket Number, store the data safely, and prevent one Requester from viewing another Requester's ticket.

Because login will be introduced in the next sprint, provide a temporary Development Requester Selection screen as “user login screen” for test purposes. The student must first choose one of several seeded Requesters. The selected Requester becomes the current testing context for creating tickets, viewing My Tickets, opening Ticket Detail, and managing attachments.

Establish a consistent Zen Green Theme and reusable form, list, badge, validation, loading, empty, error, and responsive-layout conventions that later screens can reuse.”

4. Engineering Contract for Sprint 2

For this course, an engineering contract is the sprint specification plus the evidence required to prove completion. Students must prepare the contract before implementation and keep it updated throughout the sprint. The coding agent may report “done” only when the contract is satisfied and the required tests pass.

4.1. What the Engineering Contract Must Cover

The Sprint 2 engineering contract must define and verify:

- the Development Requester Selection behavior and its limitation as a testing mechanism rather than real authentication;
- the Create Ticket, My Tickets, Requester Ticket Detail, and Attachment workflows;
- the required business rules, validation, ownership checks, loading and error states, and responsive behavior;
- the data model and API contract needed to support the sprint;
- the Zen Green UI specification and reusable component rules;
- the acceptance criteria and planned automated tests; and
- the Definition of Done, including peer review, documentation, visual inspection, and passing tests in the final `main` branch.

4.2. Explicitly Excluded from Lab 2

The following are outside the scope of Lab 2:

- **Authentication and security:** login, logout, passwords, password hashing, sessions, tokens, authenticated identities, and real role-based authorization. The Development Requester selector is only a testing mechanism and must not be treated as secure authentication.
- **IT Staff workflow:** IT Staff user dashboard and queue, claiming or reassigning tickets, changing IT Priority, and other ticket-owner functions.
- **Ticket collaboration and work tracking:** Public Comments, Internal Notes, and Actions Taken.
- **Ticket lifecycle after creation:** status changes beyond the initial **New** status, including resolution confirmation, resolving, closing, reopening, or cancelling tickets.
- **Administration functions:** Administrator management of users, Requesters, roles, and reference data.

4.3. Required Ticket Behavior and Business Rules (BR)

The stakeholder has provided the required product behavior, but not a complete engineering specification. Before implementation, work with an AI agent or LLM to identify, refine, and document the complete set of business rules in docs/lab-02/specification.md.

The rules must be numbered BR-01, BR-02, and so on. Each rule must be clear, testable, and consistent with the stakeholder request. The following are examples of mandatory rules:

BR ID	Example Mandatory Business Rule
BR-01	The official Ticket Number is generated by the backend and must be unique.
BR-02	A new Ticket begins with Current Status New.
BR-03	Lab 2 uses a Development Requester selector instead of login. The selected identity is for testing only and is not authentication.

Students must identify the remaining rules needed to cover at least:

- Ticket defaults and system-generated values;
- Requester selection and switching;
- ticket ownership;
- search, filtering, sorting, and pagination;
- validation and duplicate-submission prevention;
- failure behavior and data retained after errors;
- attachment upload, download, and soft removal;
- inactive Requesters;
- empty and no-results states;
- Ticket Detail access; and
- the transition to real authentication in Lab 3.

This handout does not provide a complete list. Students are expected to discover missing rules, resolve ambiguities, and approve the final rules before coding.

4.4. Required Fields and Validation

The Create Ticket screen must capture or display at least the following information:

- Ticket Number; Ticket Date; Requester; Category; Related System; Ticket Summary;
- Requested Priority; Description; and Attachments.

Some values are system-generated or read-only. Others are entered or selected by the Requester. Students must define in specification.md which fields are required, which are editable, the validation limits, trimming rules, allowed values, frontend and backend validation, and the behavior after failure.

Example decisions:

Field	Example Decision
Ticket Number	Read-only and generated after successful creation.
Ticket Summary	Required; define and justify suitable length constraints.
Description	Required; define and justify suitable length constraints.

The application must provide a temporary Development Requester selector as “logged-in user” before the ticket screens can be used. It is a testing mechanism, not a login screen.

Students must define the selector behavior in specification.md and ui-spec.md, including how active Requesters are loaded, how the selection is stored, how the current Requester is shown, how Requester switching works, and what happens when loading fails or no active Requesters exist.

4.5. Attachment Rules

The following attachment constraints are fixed:

- Allowed types: JPG/JPEG, PNG, WEBP, and PDF.
- Maximum size: 5 MB per file.
- Maximum active attachments: five per Ticket.
- Removal must be implemented as soft removal.
- Removed files must not be downloadable or previewed.

Students must define the remaining attachment rules in specification.md, including:

- required metadata;
- safe filename and storage behavior;
- upload failure behavior;
- removal permissions;
- confirmation and removal-reason requirements;
- behavior when a Ticket is created but attachment upload fails;
- display of removed Attachments;
- preview and download behavior; and
- the transaction or compensation strategy.

Example rule: A removed Attachment remains visible as metadata but cannot be downloaded.

5. Required Database Increment

The PostgreSQL design must support at least these concepts: Development Requester, Ticket, Attachment, Category, and Related System.

Students must determine and document the table and model names, fields and data types, primary keys, foreign keys, nullability, unique constraints, indexes, enums or reference tables, timestamps, soft-removal fields, and migration decisions. The final design must appear in specification.md and in the Prisma schema.

Example only, not a complete schema:

Concept	Example Information
Ticket	ID, official Ticket Number, Requester relationship, Category relationship, Summary, Current Status, and timestamps.

5.1. Required relationships

The database must support these conceptual relationships:

- one Requester may own many Tickets;
- one Ticket belongs to one Requester;
- one Ticket may contain many Attachments;
- one Category may be used by many Tickets; and
- one Related System may be used by many Tickets.

Students must translate these relationships into an appropriate Prisma design and document any additional relationships needed for attachment upload and removal metadata.

5.2. Required indexes or constraints

Students must consider and document:

- which fields must be unique;
- which relationships require foreign keys;
- which fields are optional;
- which fields are frequently searched, filtered, or sorted;
- which indexes are justified;
- how soft removal is represented;
- how the selected Requester is associated with a Ticket; and
- how the schema can evolve in Lab 3 when authentication is introduced.

At least one database-design decision must be justified in specification.md.

5.3. Required Seed Data

The seed must be safe to run repeatedly without creating duplicates. It must include:

- the four required Ticket Categories: Account and Access, Hardware, Software, and Network;
- a least six realistic Related Systems, such as Email, Campus Wi-Fi, VPN, LEB2 App, Grade Submission App, Printer, and Corporate Laptop. Related Systems identify the specific service, application, device, or platform affected by the ticket.
- at least four active Development Requesters; and
- at least one inactive Development Requester.

Students may choose realistic Requester names and email addresses. The inactive Requester must not appear in the Development Requester selector.

6. Required REST API Contract

The REST API must support at least these capabilities:

- retrieve active Categories;
- retrieve active Related Systems;
- retrieve active Development Requesters;
- create a Ticket;
- retrieve the selected Requester's Tickets;
- retrieve one owned Ticket;
- upload an Attachment;
- retrieve Attachment metadata;
- download an active Attachment; and
- soft-remove an Attachment.

Students must define in docs/lab-02/api-spec.md the endpoint paths, HTTP methods, parameters, request and response shapes, validation behavior, pagination metadata, ownership checks, safe errors, and status codes.

Example endpoint: POST /api/tickets. Possible purpose: create one validated Ticket for the selected Development Requester. Students must complete the request body, response body, validation rules, and error cases.

6.1. Required ticket-list query behavior

The Ticket-list API must support search, filtering, sorting, and pagination. Students must decide and document the query-parameter names, searchable fields, filterable fields, sortable fields, default and secondary sorting, page numbering, permitted page sizes, invalid-parameter behavior, and response metadata.

Partial example only:

```
GET /api/tickets?search=laptop&page=1
```

The full query contract must be designed by the student.

6.2. Partial API Examples

The following JSON is an incomplete illustration of the expected style. Students must define the final request and response contracts in api-spec.md.

Partial Example JSON

```
{ "requesterId": 1, "summary": "Laptop battery drains quickly", "requestedPriority": "MEDIUM" }
```


6.3. API Contract Decisions

For every capability, students must document the successful response, validation failures, ownership failures, missing-resource behavior, and safe unexpected-error behavior. The contract must be internally consistent and traceable to Acceptance Criteria and tests.

6.4. Expected HTTP Statuses

Students must identify appropriate HTTP status codes for successful and unsuccessful operations. Examples:

Status	Example Use
200	Successful retrieval.
201	Resource created successfully.
400	Invalid input.

Students must identify additional statuses needed for missing resources, unsupported file types, oversized uploads, ownership failure, conflicts, unexpected server errors, and other documented API cases.

7. Zen Green Theme UI Specification

Lab 2 establishes reusable presentation rules. Later labs must reuse these styles rather than inventing a new visual system for every screen. Students may make modest aesthetic improvements, but the interface must remain recognizably consistent with this specification.

Token / Element	Required Style
Primary green	#006B3C for app header, primary actions, and strong emphasis.
Secondary green	#0B7A46 for active tabs, focus accents, links, and hover states.
Pale green	#EAF6EF for selected, success, and subtle section emphasis.
Page background	#F5F7F6 or similarly quiet near-white.
Surface / cards	White with subtle border and restrained shadow.
Text	Dark charcoal-green, not pure black, for comfortable reading.
Editable field	White background with clear neutral border.
Read-only field	Soft gray-green or warm ivory shading that is clearly distinct but still readable.
Error	Dark red text and border; message appears immediately below the field.
Warning	Amber callout or badge; do not use warning color as ordinary decoration.
Success	Green confirmation with readable text and no reliance on color alone.

8. Required Application Navigation

- TokTickIT application identity;
- My Tickets navigation;
- Create Ticket navigation;

- development Requester identity display;
- clear active-page indication; and
- responsive mobile navigation.

8.1. Required Development Requester Selection Screen

TikTickIT My Tickets Create Ticket Profile

Development Requester Selection

Select Development Requester

Choose a development requester to simulate the current requester context for Lab 2.
This is for testing only and is not a login screen.

Development Requester *

Jennifer Anderson

Only active development requesters are shown.

Authentication coming in Lab 3

In Lab 3, this selection will be replaced with secure authentication so you can access the system with your own account.

Cancel Continue

Required elements:

- TokTickIT title;
- short explanation that this selector is used for Lab 2 testing only;
- Development Requester dropdown;
- active Requesters loaded from PostgreSQL;
- Continue button;
- loading state;
- empty state if no active Requesters exist;
- safe API-failure state;
- keyboard-accessible form controls; and
- responsive Zen Green styling.

Suggested explanatory text:

Select a Development Requester to test requester-specific ticket behavior. This is not a login screen. Authentication and role-based access will be introduced in Lab 3.

After selection:

- the application shell displays the Requester name;
- a Change Requester action is available;
- requester-specific data is reloaded whenever the selection changes.

8.2. Required Ticket Screen (Create Mode) Layout

Students must design the exact responsive Create Ticket layout in ui-spec.md. It must include all required Ticket information, make system-generated and read-only values visually distinct, give Summary and Description sufficient space, place Attachments logically, and provide a clear primary submission action. Example arrangement: system-generated fields near the top; classification fields grouped together; Summary and Description given sufficient width; Attachments below the main fields; and primary and secondary actions at the bottom. Students may improve the arrangement while remaining consistent with the Zen Green design. Figure 1 shows an example of the Ticket UI screen.

8.3. Component Rules

- Labels appear above controls and use consistent font weight and spacing.
- Required fields show a red asterisk. The asterisk does not replace the validation message.
- Inputs use one consistent height. Multiline Description is taller and resizable only when it does not break the layout.
- Buttons include visible text; icons may support but must not replace unclear text.
- Every icon-only control requires an accessible label and tooltip.
- Disabled controls must be visually distinct and cannot be activated.
- Focus indicators must remain visible for keyboard users.
- The Submit button shows a busy state and is disabled while the request is being processed.
- Validation messages appear near the associated field, not as one mysterious error at the top only.
- The success state clearly displays the generated Ticket Number and next action.

8.4. Required My Tickets Screen Layout

The My Tickets screen must allow the selected Requester to find and open their Tickets. Students must define the exact arrangement in ui-spec.md. The design must include search, suitable filters, sorting, pagination, a Create Ticket action, and meaningful loading, empty, no-results, and failure states.

The desktop and smaller-screen representations may differ, but both must remain clear and usable.

My Tickets

Create Ticket

Profile

My Tickets

View and track all of your support requests.

Clear Filters

Create Ticket

Search by ticket number or summary...

Category

All Categories

Requested Priority

All Priorities

IT Priority

All Priorities

Current Status

All Statuses

Ticket No.	Created Date	Summary	Category	Requested Priority	IT Priority	Current Status	Ticket Owner	Last Updated
TKT-2025-001234	May 12, 2025 09:14 AM	Laptop battery drains quickly	Hardware	Medium	Medium	In Progress	Michael Brown	May 13, 2025 10:30 AM
TKT-2025-001233	May 12, 2025 08:02 AM	Cannot connect to VPN	Network	High	High	Open	Sarah Johnson	May 13, 2025 09:45 AM
TKT-2025-001232	May 11, 2025 04:45 PM	Email not syncing on mobile	Software	Medium	Medium	In Progress	David Lee	May 12, 2025 03:20 PM
TKT-2025-001231	May 11, 2025 11:30 AM	New employee setup request	Access	Low	Low	Resolved	Jennifer Anderson	May 12, 2025 11:05 AM
TKT-2025-001230	May 10, 2025 02:10 PM	Printer keeps showing offline	Hardware	Medium	Low	Open	Michael Brown	May 11, 2025 10:15 AM
TKT-2025-001229	May 10, 2025 10:08 AM	Request access to SharePoint	Access	Low	Low	Pending	Sarah Johnson	May 10, 2025 10:08 AM
TKT-2025-001228	May 9, 2025 03:22 PM	Outlook freezing intermittently	Software	High	Medium	In Progress	David Lee	May 11, 2025 01:40 PM
TKT-2025-001227	May 9, 2025 09:15 AM	Docking station not detected	Hardware	Medium	Medium	Resolved	Michael Brown	May 9, 2025 02:30 PM

Showing 1 to 8 of 42 tickets

Previous

1

2

3

4

5

...

6

Next

The list must show enough information for a Requester to identify, understand, and open a Ticket. Students must decide and justify the final columns or card fields.

Example fields include Ticket Number, Summary, Category, Current Status, and Last Updated. The example is not a complete mandatory column list.

8.5. Required Requester Ticket Screen (View Mode) Layout

The Requester Ticket Detail screen must present the current Ticket information as read-only and provide the required Attachment functions. Students must determine the exact field grouping, responsive arrangement, attachment presentation, and navigation in ui-spec.md.

The design must clearly distinguish current Ticket information from Attachment actions and must not implement Public Comments, Internal Notes, Actions Taken or later status-workflow features.

8.6. Required Screen Modes and User Feedback

Students should identify the main screen modes (view/create/edit mode) and ensure that validation, success, failure, and empty-result feedback are handled clearly in the UI and covered by appropriate tests.

-11-

8.7. Responsive Requirements

Viewport	Required Behavior
Desktop ≥ 992 px	Multi-column layout as specified; content centered with a sensible maximum width.
Tablet 768-991 px	Two-column layout where practical; Summary and Description receive enough width.
Mobile < 768 px	Fields stack vertically; buttons remain touch-friendly; no horizontal page scrolling.
All sizes	No clipped labels, overlapping messages, hidden buttons, or unreadable attachment names.

8.8. UI Style Checking

- automated assertions for required CSS classes, field states, labels, asterisks, messages, and button behavior;
- Playwright screenshots at desktop, tablet, and mobile viewport sizes;
- a short visual checklist confirming no clipping, overlap, unintended horizontal scrolling, inconsistent field styling, or missing states;
- comparison against ui-spec.md and the approved illustrations for Create Ticket, My Tickets, and Ticket Detail rather than personal memory;
- visual inspection of desktop table and mobile ticket-card or responsive-table behavior;
- badge consistency for Requested Priority, IT Priority, and Current Status;
- checks that filters, pagination, attachment controls, and empty states remain usable at all viewport sizes.

8.9. Spec DD Deliverable

Spec DD means that implementation is driven by an explicit, version-controlled specification. Students must create the following file before asking the coding agent to implement the feature:

Required file

docs/lab-02/specification.md

The instructor provides the requirements in this handout. The student's task is to transform them into a concise, internally consistent engineering specification for their implementation. Do not copy the entire handout. Resolve implementation choices and identify assumptions.

8.10. Required specification.md Sections

Section	What the Student Must Provide
1. Sprint Goal	One short paragraph stating the delivered value.

Section	What the Student Must Provide
2. Stakeholder Request	A concise interpretation in the student's own words.
3. Scope	Included and explicitly excluded work. Include Create Ticket, My Tickets, Requester Ticket Detail, attachment lifecycle, search, filtering, sorting, pagination, and ownership protection. Explicitly exclude authentication and IT Staff workflow.
4. Functional Requirements	Numbered statements such as FR-01, FR-02, and so on. Include ticket creation, ticket listing, Ticket Detail retrieval, attachment addition, attachment soft removal, search, filters, sorting, pagination, and Requester ownership behavior.
5. Business Rules	Numbered rules such as BR-01. Include defaults, ownership, validation, and attachment rules.
6. UI Specification Summary	Screen structure, controls, states, responsive behavior, and reference to ui-spec.md. Include the application shell, Create Ticket, My Tickets, Ticket Detail, attachment states, badges, list/card behavior, and responsive rules.
7. Data Changes	Models, fields, relationships, enums, indexes, and migration decisions.
8. API Contract	Endpoints, request/response shapes, validation, statuses, and errors. Include creation, paginated list retrieval, owned-detail retrieval, attachment upload, attachment metadata, download, and soft removal.
9. Acceptance Criteria	Observable, testable criteria such as AC-01.
10. Definition of Done	Checklist covering implementation, tests, UI, review, documentation, and demonstration.
11. Assumptions and Decisions	Only meaningful choices not already fixed by the handout.

8.11. Example Acceptance Criteria

Students must create a complete set of numbered Acceptance Criteria in specification.md. The examples below demonstrate the expected Given-When-Then style:

ID	Example Criterion
AC-01	Given valid Ticket data, when the Requester submits the form, then one Ticket is saved and the official Ticket Number is displayed.
AC-02	Given no Development Requester is selected, when the user attempts to open My Tickets, then the Requester Selection screen is shown.
AC-03	Given Requester B is selected, when a Ticket belonging to Requester A is requested, then the Ticket data is not returned.

Students must add enough criteria to cover creation, validation, ownership, search, filtering, sorting, pagination, Attachments, soft removal, Requester switching, empty and failure states, responsive behavior, and accessibility. Every Acceptance Criterion must map to at least one planned test.

9. Test DD and TDD Deliverable

Test DD means that required evidence is planned from the specification before implementation is declared complete. TDD means students use those scenarios to write failing tests, implement the smallest correct behavior, and refactor while keeping the tests green.

Required file

docs/lab-02/tests.md

The plan must be created before or alongside implementation. It must not be reconstructed afterward from whatever tests the coding agent happened to generate.

9.1. Required Planned-Test Table

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final
API-01	API	AC-01	Create valid ticket	201; one saved Ticket; number returned	server/tests/lab-02/tickets.api.test.ts	Pass
UI-03	UI	AC-02	Submit without Summary	Field message; API not called	client/src/.../CreateTicket.test.tsx	Pass
E2E-01	E2E	AC-01, AC-05	Complete responsive submission flow	Confirmation shows official number	e2e/lab-02/create-ticket.spec.ts	Pass

9.2. Minimum Test Coverage

The Test Plan must include tests at these levels: unit, API or integration, UI component, UI style, responsive, and E2E.

Examples:

Level	Example Test
Unit	Ticket Number generator returns the required format.
API	Valid Ticket creation returns a created response.
UI	Submit button displays a busy state.
E2E	A Requester creates a Ticket and later finds it in My Tickets.

Students must identify the remaining scenarios needed to prove all business rules and Acceptance Criteria. The final plan must include happy paths, invalid input, boundaries, ownership, failures, loading and empty states, responsive behavior, Attachment lifecycle, and multi-Requester behavior.

Every Acceptance Criterion must map to at least one planned test, and every planned automated test must identify its actual test-file path.

10. GitHub Issues and Workflow

Use the same Kanban statuses introduced in Lab 1: Backlog, Specified, Started, PR Review, Fixing, and Done. Before coding, students must decompose the sprint into a reasonable set of GitHub Issues. The Issues must collectively cover specification and test planning, data design, APIs, frontend screens, automated tests, E2E testing, visual inspection, and release integration.

Examples:

Example Issue	Possible Scope
Sprint specification and test plan	specification.md, tests.md, ui-spec.md, and api-spec.md.
Development Requester context	Database model, seed, selector, context behavior, and tests.
Ticket creation	API, UI, validation, and tests.

Students must create the remaining Issues, identify dependencies, assign suitable feature-branch names, and explain the decomposition.

10.1. Required Branch Flow

Create lab2-staging from the current main branch after Lab 1 is complete. Each Issue is implemented on its own feature branch and enters lab2-staging through a peer-reviewed Pull Request. After integration testing, open one release Pull Request from lab2-staging to main. Do not develop directly on main or lab2-staging.

11. AI Agent Rules

These labs contain substantial requirements and documentation. Students are expected to work with an AI specification agent or LLM to develop the engineering contract before coding begins. The work is then handed over to the AI Coding Agent.

11.1. AI Specification Agent Rules

Use the AI specification agent to help prepare and refine all required documents, including `specification.md`, `tests.md`, `ui-spec.md`, and `api-spec.md`. Apply Spec-Driven Development and Test-Driven Development so that the requirements, business rules, acceptance criteria, test scenarios, UI expectations, data design, and API behavior are sufficiently complete and internally consistent.

Students must review, correct, and approve every generated document. The AI specification agent assists with analysis and drafting, but the student remains responsible for the final engineering contract.

Only after the required documents are complete should they be provided to the AI coding agent for implementation.

11.2. AI Coding Agent Rules

- Give the coding agent the current specification.md, tests.md, ui-spec.md, and api-spec.md as its contract.
- Ask the agent to identify ambiguities before writing code rather than silently inventing business rules.
- Use small implementation tasks tied to one Issue and one feature branch.
- Require the agent to state which acceptance criteria and tests it completed.
- Do not accept “done” when tests are missing, skipped, flaky, or unrelated to the acceptance criteria.
- Review every changed file, dependency, database migration, generated command, and test.
- Students must be able to explain the implementation and demonstrate failure cases.

Example Coding Agent Prompts:

Prompt Name	Example Prompt
Review Contract	Read docs/lab-02/specification.md, tests.md, ui-spec.md, and api-spec.md. List ambiguities, conflicts, dependencies, and the proposed implementation order. Do not write code yet.
Create Failing API Tests	Implement the planned API tests for the current Issue first. Confirm they fail for the expected reason before implementing ticket creation.
Implement UI Increment	Implement only the Create Ticket screen and reusable Zen Green form components required by the current Issue. Preserve the API contract and do not implement My Tickets or Ticket Detail until their specification and failing tests are available.
Completion Review	Audit the implementation against every acceptance criterion and planned test. Report missing evidence, skipped tests, untested failure states, and UI-spec deviations. Do not claim completion until corrected.
Implement My Tickets	Read the My Tickets requirements, API contract, acceptance criteria, and planned tests. Implement only the Requester-owned paginated ticket list, search, filters, sorting, loading, empty, no-results, and failure states. Do not add authentication or IT Staff workflow.
Implement Requester Ticket Detail	Implement the Requester Ticket Detail and Attachment lifecycle described in the contract. Ticket header fields are read-only. Enforce ownership in the backend. Support adding, downloading, and soft-removing permitted attachments. Do not add comments, internal notes, Actions Taken, or status changes.

Prompt Name	Example Prompt
Implement Development Requester Context	Read the Development Requester requirements, business rules, acceptance criteria, UI specification, API contract, and planned tests. Implement the temporary Lab 2 RequesterUser model, idempotent seed data, active Requester API, Development Requester Selection screen, selected Requester context, and Change Requester behavior. Clearly label this as a testing mechanism, not authentication. Do not add passwords, login, sessions, roles, or Lab 3 functionality.

12. Required Repository Increment

Minimum Lab 2 structure

```
docs/lab-02/
├── specification.md
├── tests.md
├── ui-spec.md
├── api-spec.md
├── reviewer.md
└── ai-use.md

server/tests/lab-02/
├── create-ticket.api.test.ts
├── my-tickets.api.test.ts
├── ticket-detail.api.test.ts
└── attachments.api.test.ts

client/.../lab-02 tests/
├── CreateTicket.test.tsx
├── MyTickets.test.tsx
├── RequesterTicketDetail.test.tsx
└── AttachmentSection.test.tsx

e2e/lab-02/
└── requester-ticket-flow.spec.ts

artifacts/lab-02/screenshots/
├── create-ticket/
├── my-tickets/
└── ticket-detail/
```

13. Definition of Done for Lab 2

For this course, the Definition of Done has two parts:

13.1. Part 1: Product Completion

This part defines the conditions that the software increment must satisfy before the AI coding agent may report that the work is complete.

Students must work with the AI specification agent or LLM to prepare a complete Product Definition of Done in `docs/lab-02/specification.md`. It should cover areas such as:

- implementation of all approved scope;
- satisfaction of all acceptance criteria;
- passing and traceable automated tests;
- conformance to the data, API, UI, validation, and responsive specifications;
- correct handling of success, failure, and boundary cases; and
- current setup and usage documentation.

Example items include:

- All required tests pass from documented commands in the final `main` branch.
- Every acceptance criterion is linked to appropriate test evidence.
- No required test is skipped, disabled, or commented out.
- The implemented screens and APIs conform to the approved engineering contract.
- README setup and test instructions are current.

These examples are not a complete checklist. Students must identify and add the remaining product-completion conditions needed for the Development Requester workflow (“login”), Ticket creation, My Tickets, Ticket Detail, Attachments, ownership behavior, responsive UI, and error handling.

The AI coding agent should use this Product Definition of Done when deciding whether implementation is complete.

13.2. Part 2: Course Delivery Requirements

This part covers how the completed work must be delivered and reviewed for the course. It includes requirements such as:

- use of GitHub Issues and feature branches;
- Pull Requests through the required staging workflow;
- peer review and approval;
- responses to review comments;
- required repository documents; and
- submission of the specified PDF evidence.

These are course process and assessment requirements. They are checked separately from the Product Definition of Done.

14. Submit One PDF File

Submit exactly one concise PDF. To make grading consistent for approximately 200 students, use the headings “Answer Part 1” through “Answer Part 9” in this exact order. Include working links. Screenshots must be readable without extreme zoom. The submitted repository and main branch remain the source of truth.

Part	Points	Required Submission Evidence
1. Git Use with Engineering Workflow	10	- Screenshot evidence that you used the git workflow by showing your commit history in the final main branch showing you created various feature branches that were eventually merged into the staging branch and then the main branch. - Evidence you used GitHub Project and final Kanban with all Issues in Done. - Rendered reviewer.md with reviewer identity, PR links, comments given and received, responses, and approvals. - Include README and .gitignore content evidence. - Directory Structure of your repository in your IDE
2. Spec DD	5	Link to and rendered copy of docs/lab-02/specification.md. Show numbered requirements, business rules, acceptance criteria, and Definition of Done. Include one screenshot proving it existed before the main implementation PRs were completed.
3. Test DD and Traceability	10	Link to and rendered copy of docs/lab-02/tests.md. Include the planned-test table, acceptance-criterion traceability, actual test-file paths, and final pass status. Include complete unit, API, and UI passing test output from main.
4. AI Use with Reflection	5	Rendered docs/lab-02/ai-use.md that mentions the LLM you used and provides a table of 6-10 selected key prompts. Provide a very brief “My Reflection” on your AI use experience.
5. Development Requester Select Screen	0	The simulated Login screen to select who the requester user is for this session. Points for this will be included in Working Ticket Screen Create Mode.
6. Working Ticket Screen: Create Mode	10	Readable screenshots of initial, validation failure, submitting, success, API failure, and invalid-attachment states. Demonstrate that the official Ticket Number and saved values come from the backend/database. Include the Development Requester Selection screen, active-user dropdown, selected-user display, Change Requester action, loading state, and failure state. 1. Show that the Requester field is populated from the Development Requester selected before entering the application and that the saved Ticket contains the matching <code>requesterId</code>. 2. Open Create Ticket at a desktop viewport and show reference data loaded from the database. 3. Attempt an invalid submission and show field-level messages. 4. Select one valid and one invalid attachment and explain the result. 5. Stop the backend or simulate failure and show the safe error state with form values preserved.

Part	Points	Required Submission Evidence
7. Working My Tickets Screen	10	Show Requester A selected and their ticket list. Change to Requester B and demonstrate that Requester A's tickets disappear. Include search, filters, sorting, pagination, empty state, no-results state, and cross-requester access evidence.
8. Working Ticket Screen: View Mode and Attachments	5	Owned Ticket Detail, add attachment, download active attachment, soft removal with reason, retained metadata, blocked removed download, unauthorized ticket-access test. Include evidence that direct access to a Ticket or Attachment belonging to a different selected Requester is rejected.
9. Zen Green UI and Responsive Evidence	5	Rendered ui-spec.md plus desktop, tablet, and mobile screenshots. Include the completed visual checklist covering colors, editable/read-only fields, validation placement, button hierarchy, clipping, overlap, and horizontal overflow.

To make grading faster and more consistent, **you must your PDF using the following format:**

Answer Part 1:

[Place your content here]

Answer Part 2:

[Place your content here]

...

Answer Part 9:

[Place your content here]

15. Appendix A. Suggested specification.md Template

You are not expected to write the entire `specifications.md` file from scratch. Instead, work with the AI Spec agent or LLM to refine it until the requirements, business rules, acceptance criteria, and test scenarios are sufficiently detailed and complete before implementation begins. The goal is to make the specification clear enough that the coding agent can complete the feature correctly in one focused implementation pass, with minimal rework.

Template

Lab 2 Sprint Engineering Specification

1. Sprint Goal

2. Stakeholder Request Interpretation

3. Scope

Included

Excluded

```
## 4. Functional Requirements
- FR-01 ...
## 5. Business Rules
- BR-01 ...
## 6. UI Specification Summary
## 7. Data Changes
## 8. API Contract
## 9. Acceptance Criteria
- AC-01 Given..., when..., then...
## 10. Definition of Done
## 11. Assumptions and Decisions
```

16. Appendix B. Suggested tests.md Template

You are not expected to write the entire `tests.md` file from scratch. Work with the AI agent or LLM to identify a complete set of unit, API, UI, responsive, visual, and end-to-end test scenarios based on the specification and acceptance criteria. You must then review, refine, and approve the proposed tests before implementation begins. The goal is to ensure that every important requirement has clear test coverage so the coding agent can use the test plan as evidence that the feature is truly complete, rather than simply claiming that it works.

Template

```
# Lab 2 Test Plan and Results

## 1. Test Strategy
## 2. Planned Tests
[Insert planned-test table]
## 3. Acceptance-Criterion Traceability
[Insert AC-to-test matrix]
## 4. Responsive and Visual Checklist
## 5. Test Commands
## 6. Final Results
## 7. Known Limitations or Deferred Tests
```

17. Appendix C. Suggested ui-spec.md Checklist

You are not expected to design the entire `ui-spec.md` file from scratch. Work with the AI agent or LLM to turn the required Create Ticket, My Tickets, Ticket Detail, Attachment, list, table, card, badge, pagination, form-state, responsive, and accessibility behavior, visual style, responsive rules, and accessibility expectations into a complete UI specification before implementation begins.

Provide the agent with the approved sample screen and UI style reference so it can infer the intended visual direction, including the Zen Green Theme color palette, spacing, field states, button hierarchy (Primary, secondary, tertiary, destructive, disabled, and busy button styles), and overall form styling. However, do not rely on the image alone. The final ui-spec.md must explicitly document the chosen color tokens, typography, spacing, component states, validation placement, responsive behavior, accessibility rules, and screenshot-based visual checks.

You must review and approve the generated UI specification. The goal is to make the expected interface precise enough that the coding agent can implement it consistently and that the finished screen can be checked against clear visual and functional criteria.

- Color tokens and their intended use.
- Typography and spacing.
- Editable, read-only, invalid, disabled, and focused controls.
- Required-field marker and validation-message placement.
- Button hierarchy and busy state.
- Attachment selection and error presentation.
- Initial, loading, validation, submitting, success, and failure states.
- Desktop, tablet, and mobile layout rules.
- Accessibility labels, keyboard focus, and non-color indicators.
- Visual inspection checklist and screenshot paths.
- Application shell and active navigation.
- Ticket-list columns and mobile representation.
- Search, filters, sort, clear-filters, and pagination controls.
- Priority and status badge rules.
- Empty-list versus no-results presentation.
- Requester Ticket Detail read-only layout.
- Active, uploading, invalid, removed, and unavailable attachment states.
- Desktop table and mobile card or responsive-table behavior.
- Screenshot paths for Create Ticket, My Tickets, and Ticket Detail.